

Lab 03 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Drie gedragingen, één regel

Antwoord. De regel: **een container leeft precies zolang zijn PID 1.** `alpine` heeft als standaardcommando `/bin/sh`; zonder standaardinvoer leest de shell een bestandseinde en stopt meteen → de container stopt. `nginx` blijft op de voorgrond en geeft nooit de hand terug → de container leeft, en blokkeert je terminal omdat je geen `-d` zei. `-it alpine sh` geeft de shell een open invoer en een terminal → hij wacht op je commando's, dus de container leeft tot je `exit` typt.

Waarom. De engine start alleen een proces in namespaces en wacht op het einde ervan. Er bestaat geen begrip "dienst": wat de container in leven houdt, is een proces dat niet eindigt.

Nuance. `podman run nginx` zonder `-d` betekent niet dat nginx "anders" draait: het is identiek, alleen je terminal is aan zijn uitvoer gekoppeld. `Ctrl+C` stuurt dan `SIGINT` naar PID 1 en stopt het.

Voorbeeld.

```
podman run alpine;           podman ps -a -l --format '{{.Status}}'    # Exited (0)
podman run -d nginx:alpine;  podman ps -l --format '{{.Status}}'    # Up
podman run -it alpine sh -c 'echo "ik leef zolang je wilt"; exit 7'; echo $?    # 7
```

Vraag 2 — De `&` die doodt

Antwoord. Het script is PID 1. `java ... &` start Java op de achtergrond en geeft meteen de hand terug; `echo` wordt uitgevoerd; het script bereikt zijn einde en stopt met code `0`. PID 1 doodt, dus de kernel doodt al de rest van de namespace, Java inbegrepen. Correctie: Java op de voorgrond starten **en** als laatste regel, met `exec`:

```
#!/bin/sh
echo "API gestart"
exec java -jar /app/api.jar
```

Waarom. `exec` vervangt de shell door Java, dat PID 1 wordt: het leeft zolang het wil en ontvangt `SIGTERM` rechtstreeks. Zonder `exec` maar zonder `&` zou het script op Java wachten (de container zou leven) maar PID 1 blijven vóór Java — en `SIGTERM` niet doorgeven (vraag 3 van lab 04).

Nuance. Code `0` is misleidend: alles "verliep goed" vanuit het oogpunt van het script. Het is een voorbeeld van een container die faalt zonder fout — de *restart policies* `on-failure` zouden hem niet eens herstarten.

Voorbeeld.

```
podman run --rm -v "$PWD":/s alpine /s/demarrage-casse.sh    # komt meteen terug
podman run -d --name ok -v "$PWD":/s alpine /s/demarrage-correct.sh && podman top ok    # sleep
als PID 1
```

Vraag 3 — Tien seconden en 137

Antwoord. `sleep` is PID 1, en de Linux-kernel laat PID 1 elk signaal negeren waarvoor het geen handler heeft geïnstalleerd. `sleep` installeert er geen: `SIGTERM` wordt genegeerd. Podman wacht de respijtp periode (10 s) af, kondigt aan dat het overgaat naar `SIGKILL` — dat niemand kan negeren — en het proces sterft gedood: code `128 + 9 = 137`. `143` (`128 + 15`) verschijnt alleen wanneer het `SIGTERM` is dat het proces effectief beëindigd heeft.

Waarom. Die bescherming van PID 1 bestaat opdat een onhandige `kill -TERM 1` geen hele machine neerhaalt. In een container keert ze zich tegen jou.

Nuance. Dit is niet eigen aan `sleep`: elk programma zonder `SIGTERM`-handler gedraagt zich zo als PID 1 — ook een shells script, of een `java` gestart achter een shell. De waarschuwing die Podman toont (`resorting to SIGKILL`) is waardevol: Docker doodt in stilte.

Voorbeeld.

```
podman run --rm alpine sh -c 'kill -TERM 1; echo overleefd'      # "overleefd": PID 1 negeerde
                        zijn eigen TERM
podman run -d --name v alpine sleep 300; time podman stop v    # 10 s, code 137
```

Vraag 4 — Wat `--init` verandert

Antwoord. `--init` plaatst `podman-init` (een binary van enkele KB, `catatonit`) als PID 1; `sleep` wordt zijn kind, PID 2. `podman-init` kan twee dingen: signalen doorgeven aan zijn kind en zombies opruimen. Bij de `stop` ontvangt het `SIGTERM` en geeft het door aan `sleep`, dat — geen PID 1 meer — de standaardactie ondergaat: sterven. Code `143`, onmiddellijk. `podman exec` wacht `ps` toont `1 podman-init` en dan `2 sleep`.

Waarom. De bescherming van de kernel geldt alleen voor PID 1. Door je programma naar PID 2 te verplaatsen, krijgt het een normaal gedrag tegenover signalen terug.

Nuance. `--init` is een pleister: het maakt je applicatie niet in staat tot een nette stop, het maakt ze alleen *netjes te doden*. Een Spring Boot-API behandelt `SIGTERM` zelf; ze heeft geen `--init` nodig, ze moet het **ontvangen** (exec-vorm). `--init` blijft nuttig voor images die meerdere processen starten en zombies produceren.

Voorbeeld.

```
podman run --rm --init alpine ps -o pid,comm                  # 1 podman-init, 2 ps
```

Vraag 5 — `-i` zonder `-t`, `-t` zonder `-i`

Antwoord. `-i` houdt `stdin` open en verbonden met je toetsenbord; `-t` kent een pseudo-terminal toe (prompt, echo, toetsafhandeling). `podman run -t alpine sh`: je ziet een prompt, maar `stdin` is niet verbonden — je toetsaanslagen gaan nergens heen, `ls` doet niets, en de container blijft daar hangen tot je hem vanuit een andere terminal doodt (`podman rm -f -t 0`). `podman run -i alpine sh`: geen prompt of echo, maar wat je typt wordt doorgegeven: `ls` wordt uitgevoerd en toont zijn resultaat, zonder comfort.

Waarom. Het zijn twee onafhankelijke kanalen: `-i` betreft de gegevensstroom, `-t` de presentatie. Een shell heeft alleen `-i` nodig om te werken; hij heeft `-t` nodig om aangenaam te zijn.

Nuance. `-i` alleen is de vorm voor scripts: `echo "SELECT 1" | podman exec -i db psql -U app` werkt, terwijl het met `-t` zou falen (`the input device is not a TTY`). Een klassieke CI-bug.

Voorbeeld.

```
echo 'echo "ontvangen: $((6*7))"' | podman run -i --rm alpine sh      # ontvangen: 42 – zonder
prompt                                                                # prompt "/ #", Ctrl+D om
podman run -it --rm alpine sh                                         # prompt "/ #", Ctrl+D om
eruit te gaan
```

Vraag 6 — `attach` en `Ctrl+C`

Antwoord. `attach` heeft zijn terminal aan de stromen van **PID 1** gekoppeld — de API zelf. `Ctrl+C` stuurde `SIGINT` naar dat proces, dat stopte; de container stierf mee. De twee juiste manieren: `podman logs -f mijn-api` (leest de logs die `common` opving, `Ctrl+C` stopt alleen de weergave) of `podman exec -it mijn-api sh` (nieuw proces, zonder effect op PID 1).

Waarom. `attach` maakt niets aan: het verbindt je terminal opnieuw met de bestaande pijpen van het hoofdproces, signalen inbegrepen. Het is precies wat je zou hebben door de container op de voorgrond te starten.

Nuance. Er is een uitweg: `Ctrl+P` `Ctrl+Q` koppelt los zonder te stoppen (als de container met `-it` gestart is), en `podman attach --sig-proxy=false` verhindert het doorgeven van signalen. Maar het echte antwoord is `attach` niet te gebruiken om logs te lezen.

Voorbeeld.

```
podman logs -f --tail 20 mijn-api      # Ctrl+C: de container draait voort
podman attach --sig-proxy=false mijn-api # Ctrl+C wordt niet doorgegeven
```

Vraag 7 — 137, 143, 127

Antwoord. `api` (137): gedood door `SIGKILL` — ofwel een `stop` waarvan de respijtperiode verstreek, ofwel de OOM killer. Bevestigen: `podman inspect --format '{{.State.OOMKilled}}' api`, dan `podman events --since 1h | grep api` om te zien of er een `stop` was. `worker` (143): ontving `SIGTERM` en stopte — een vrijwillige stop (uitrol, `podman stop`); bevestigen met `podman events` of `journalctl`. `batch` (127): het commando werd niet gevonden — de applicatie is nooit gestart (fout in image of `CMD`). Bevestigen: `podman logs batch` (melding `executable file not found`) en `podman inspect --format '{{json .Config.Cmd}}' batch`.

Waarom. Boven 128 is de code `128 + signaalnummer`. Daaronder is het de code die het programma koos — of de shell/runtime wanneer het programma niet gestart kon worden.

Nuance. Een 137 met `OOMKilled: false` en zonder `stop` in de gebeurtenissen kan komen van een handmatige `kill -9` of van een orchestrator. En de 143 van `worker` op hetzelfde moment als de 137 van `api` wijst op een gegroepeerde stop waarbij `api` niet netjes kon stoppen: het symptoom van een shell ervoor (lab 04).

Voorbeeld.

```
podman inspect --format 'oom={{.State.OOMKilled}} einde={{.State.FinishedAt}}' api
podman events --since 1h --filter container=api
```

Vraag 8 — Logs in een bestand

Antwoord. `podman logs` geeft alleen terug wat `common` heeft opgevangen op `stdout / stderr` van PID 1. Door naar een bestand te schrijven, omzeilt de applicatie dat kanaal: er wordt niets opgevangen. De map op de host mounten maakt het bestand leesbaar, maar blijft een slecht antwoord: de logs ontsnappen aan de tooling (`podman logs`, `journald`, verzamelagents), elke container verzint zijn eigen pad, rotatie wordt niet beheerd, en een verwijderde container laat weesbestanden achter.

Waarom. Het containermodel behandelt logs als een **stroom**: de engine vangt ze op, de tooling stuurt ze door (bestand, journal, Loki, Elastic). Een bestand in de container is een lokale toestand, in strijd met de wegwerpaard van de container.

Nuance. Spring Boot logt standaard naar de console: het volstaat `logging.file.name` **niet** te definiëren. Als een bestandsformaat opgelegd wordt, is de oplossing een *sidecar* of een agent die de stroom leest, geen mount.

Voorbeeld.

```
podman run -d --name l alpine sh -c 'echo zichtbaar; echo onzichtbaar > /tmp/app.log; sleep 100'
podman logs l                                # zichtbaar
podman exec l cat /tmp/app.log                # onzichtbaar – alleen door binnen te gaan
```

Vraag 9 — `stop / start` tegenover `rm / run`

Antwoord. Na `stop` en dan `start`: de gegevens zijn **bewaard** — de schrijflaag van de container bestaat nog, PostgreSQL vindt zijn bestanden terug. Na `rm` en een nieuwe `run`: de gegevens zijn **verloren** — `rm` heeft de schrijflaag vernietigd, en de nieuwe container vertrekt van de image.

Waarom. `stop` werkt alleen op het proces; de container (configuratie + laag) blijft. `rm` verwijdert het containerobject, laag inbegrepen.

Nuance. De image `postgres` declareert een `VOLUME`: de gegevens gaan naar een anoniem volume dat de `rm` overleeft maar nergens meer aan gekoppeld is — in de praktijk niet te recupereren. Het benoemde volume (lab 06) is de enige echte persistentie.

Voorbeeld.

```
podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c 'create table t(x int)'
podman stop db && podman start db && podman exec db psql -U postgres -c '\dt'    # t is er
podman rm -f -t 0 db && podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c '\dt'                                       # niets meer
```

Vraag 10 — `--restart=always` en de reboot

Antwoord. Onder Docker herleest de daemon bij het opstarten het beleid en herstart hij de containers. Onder Podman is er geen daemon: `--restart=always` wordt toegepast door `common` zolang de container *in een levende sessie bestaat*, maar na een reboot draait er niets om het beleid te herlezen. De Podman-manier: een **Quadlet**-bestand (`/etc/containers/systemd/api.container`, of `~/config/containers/systemd/` in rootless-modus)

dat de container beschrijft, en `systemctl enable --now api` — systemd start hem bij het booten en herstart hem bij falen.

Waarom. Podman koos ervoor geen dienstbeheerder opnieuw uit te vinden: Linux heeft er een, systemd, met zijn afhankelijkheden, zijn logs en zijn start bij het booten. Een Podman-restart policy dekt alleen het leven van een sessie.

Nuance. In rootless-modus heb je bovendien `loginctl enable-linger <gebruiker>` nodig opdat de diensten van de gebruiker starten zonder open sessie. Op een WSL-werkpost is dat zelden nodig: ontwikkelcontainers hoeven geen reboot te overleven.

Voorbeeld.

```
# ~/.config/containers/systemd/api.container
[Container]
Image=registry.intern/mijnapp/api:1.4.2
PublishPort=8080:8080
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start api && systemctl --user status api
```

Vraag 11 — De logs van de eerste poging

Antwoord. In `podman logs <container>`: de logs **stapelen zich op** op dezelfde container bij elke herstart, de eerste poging staat bovenaan. `podman restart` wist ze evenmin — maar je verliest `.State.ExitCode` en `.State.FinishedAt` van de laatste uitvoering, en vooral gaat de container opnieuw in zijn lus. Kijk eerst.

Waarom. Een automatische herstart start **dezelfde** container opnieuw (zelfde ID, zelfde schrijflaag, zelfde logbestand), hij maakt geen nieuwe aan. `podman events` geeft daarbovenop de exacte chronologie (`died` , `restart`).

Nuance. Een `podman rm` (of `--rm`) verwijdert alles, logs inbegrepen. En een container die in een lus herstart, kan omvangrijke logs produceren: `--tail` en `--since` zijn je vrienden.

Voorbeeld.

```
podman logs --timestamps onstabiel | head -20          # de eerste uitvoering
podman events --since 10m --filter container=onstabiel
```

Vraag 12 — Van minst naar meest ingrijpend

Antwoord. (1) `podman inspect`: leest metadata, geen effect — configuratie, toestand, OOM, host-PID. (2) `podman logs`: leest wat `common` al opving — wat de applicatie over zichzelf zegt. (3) `podman stats`: leest de cgroups — echte CPU, geheugen, I/O, zonder de container aan te raken. (4) `podman top`: voert aan hostzijde een `ps` uit op de PID's van de container — welk proces verbruikt, welke threads. (5) `podman exec`: maakt een proces aan **in** de container — het meest ingrijpend, maar het enige dat een `jstack` of een `curl localhost:8080/actuator` toelaat.

Waarom. De eerste vier observeren van buitenaf, via de engine of de kernel; alleen `exec` wijzigt de binnenkant (een proces meer, resources verbruikt in de cgroup van de container).

Nuance. Met de host-PID die `inspect` geeft, kun je verder gaan zonder `exec`: `cat /proc/<pid>/status`, `strace -p <pid>` — aangezien de container in rootless-modus een proces van jouw gebruiker is. En een *distroless* image heeft geen shell: `exec` is er niet mogelijk (lab 05).

Voorbeeld.

```
podman stats --no-stream api
podman top api pid,pcpu,comm
podman exec api jcmd 1 Thread.print | head -50
```

Vraag 13 — API en database in dezelfde container

Antwoord. Drie gevolgen: (1) **één enkele PID 1**: je hebt een toezichthouder (`supervisord`) nodig om twee processen vast te houden, en als de database sterft, weet de container het niet — of omgekeerd, de API sterft en neemt de database mee; (2) **gekoppelde levenscyclus**: de API opnieuw uitrollen dwingt tot een herstart van PostgreSQL, met zijn verbindingen en zijn cache; (3) **resources en observeerbaarheid vermengd**: één geheugenlimiet, één vermengde logstroom, onmogelijk de API te schalen zonder de database te dupliceren.

Waarom. De container is ontworpen rond één hoofdproces waarvan het leven dat van de container is. Twee processen, dat zijn twee levenscycli in een object dat er maar één heeft.

Nuance. Podman heeft een object voor "meerdere containers die samen moeten leven": de **pod** (`podman pod create`), die netwerk en levenscyclus deelt en toch één container per proces behoudt — hetzelfde concept als Kubernetes. Dat is het correcte antwoord op de behoefte "eenvoudig starten".

Voorbeeld.

```
podman pod create --name stack -p 8080:8080
podman run -d --pod stack --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman run -d --pod stack --name api mijn-api:1.0      # bereikt db op localhost:5432
```

Vraag 14 — `--rm` en productie

Antwoord. Voor een eenmalig commando vermijdt `--rm` het opstapelen van lijken. Voor een dienst in productie vernietigt het bij het afsluiten precies wat men na een incident nodig heeft: de **logs**, de **exitcode**, de **schrijfslag** (tijdelijke bestanden, *heap dump*), en de mogelijkheid tot `podman inspect`. De container is dood en er valt niets meer te onderzoeken. De combinatie met `--restart` is tegenstrijdig door constructie: `--rm` verwijdert de container bij het afsluiten, `--restart` wil hem bij het afsluiten herstarten — je kunt niet herstarten wat je net gewist hebt. Podman weigert het expliciet.

Waarom. De `Exited`-container is het voorwerp van de autopsie. Een dienst die om 3 uur 's nachts crashte, moet om 9 uur inspecteerbaar zijn.

Nuance. Orchestrators (Kubernetes, Compose) beheren zelf het verwijderen van beëindigde containers, met een vertraging en een logretentie. `--rm` blijft perfect voor tool-containers: compilatie, databasemigratie, interactieve `psql`.

Voorbeeld.

```
podman run --rm -d --restart=always nginx:alpine
```

```
# Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"
```